# Operating system and system programming (25CS2104E)

# Linux-Based Alarm Clock Using System Programming Concepts

## Section 12 | Team 1

Review-I

Under the Esteemed Guidance of

**Raghupathi M**
**CSIT Department**

**Team Members:**

K. Laxmi Lahari – 2520090206

S. Rishil – 2520090174

S. Anjani Shreyaj – 2520090082

# Contents

- Abstract
- Introduction
- Hardware and Software requirements
- System Architecture
- Conclusion

# Abstract


- A Linux-based Alarm Clock built in C that demonstrates Operating System and POSIX concepts, not just another GUI-style application.
- Uses the alarm() system call to schedule a SIGALRM signal once the requested waiting period has elapsed.
- A custom SIGALRM handler notifies the user in the terminal the moment the alarm time is reached.
- Simple terminal interface to set, view, and cancel alarms; the process suspends with pause()/sleep() instead of busy-waiting, keeping CPU usage negligible.
- Outcome: a working alarm clock that gives hands-on understanding of process management, signals, timers, and system calls.


SIG — SIGALRM Handling

TMR — alarm() / setitimer() Timers

PID — Process Management

SYS — System Calls

# Introduction

## Problem Statement

Most alarm apps are generic GUI or mobile applications that hide the OS mechanisms behind timing and notification.

This project builds a Linux terminal Alarm Clock that explicitly demonstrates how OS concepts schedule and trigger an alarm.

The system accepts an alarm time, computes the waiting period, and notifies the user — using Linux/POSIX signals, timers, process management and system calls rather than a generic GUI or database application.

## Objectives

1. Design and implement a Linux-based terminal Alarm Clock in C.
2. Use Linux/POSIX system calls and signals to schedule and trigger alarms.
3. Demonstrate OS concepts: process management, signal handling, and timer-based events.
4. Provide a simple, reliable way to set, view, and cancel alarms via a terminal interface.

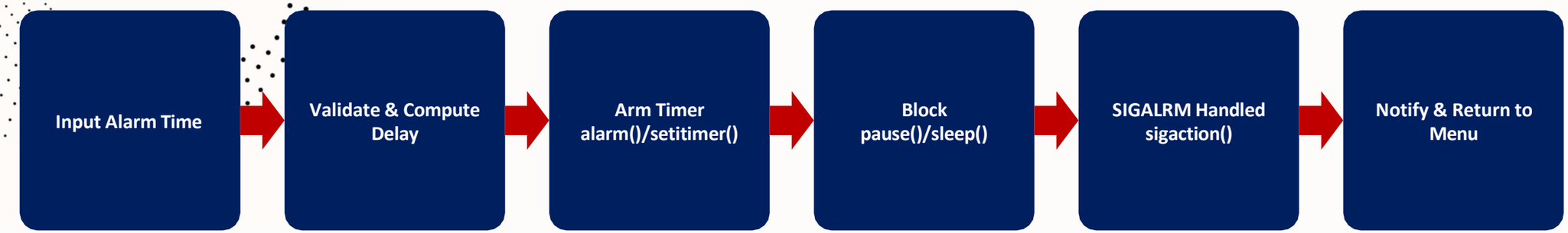
# Hardware and Software requirements

**Hardware Requirement:** Any standard PC/laptop capable of running Linux (Ubuntu recommended)

| Tool / Platform | Purpose |
|---|---|
| **Linux / Ubuntu** | OS platform providing POSIX APIs for signals, timers, processes |
| **C / C++** | Programming language used to implement the alarm clock logic |
| **GCC** | Compiler used to build the C source code |
| **Bash Terminal** | Environment used to run, test, and interact with the program |
| **Make / Makefile** | Automates the compilation and build process |
| **VS Code / gedit** | Text editor used for writing and debugging the source code |
| **Git / GitHub** | Maintains, manages, and submits the source code and docs |

# System Architecture

## Methodology Flow




## Process Management & Cancellation

- **fork()** runs the alarm wait as a background process, keeping the terminal free.
- **Process ID (PID)** identifies and tracks the running alarm process.
- **kill() / alarm(0)** cancels an alarm before it triggers.

# Conclusion

## Expected Outcome :

- A working Linux terminal Alarm Clock that lets a user set, view, and cancel one or more alarms.
- Reliable notification via SIGALRM with no busy-waiting or unnecessary CPU usage while idle.
- Hands-on understanding of how the Linux/POSIX kernel manages processes, delivers signals, and schedules timer-based asynchronous events.
- Final submission includes source code, build instructions, terminal output, and test screenshots.

## Test Cases Verified

1. Setting a single alarm
2. Setting multiple concurrent alarms
3. Cancelling an alarm before it triggers

**GitHub Repository:** github.com/rishill10/KLH-CSIT-2026-27-01-Alarm-Clock

# Thank you